

INTRODUCTION

Liam Power

WHO AM I?

WHO AM I?

- My name is **Liam Power**
- I am **20** years old
- Born and raised in **Dublin**



TWO TRUTHS AND A LIE

I worked in a prison
alongside inmates in Dublin

I have been to North Korea
and chatted with locals

I was stranded shirtless in
the Slovenian wilderness



MY COLLEGE

- I am a **3rd Year** college student at Trinity College Dublin
- I study **Business and Computer Science**

YOUR EDUCATOR

- You will be studying the exact same material that I learnt in my first year in college
- I found it tricky at times, so if you ever have a question, feel free to ask it!
- I'm not a teacher, I'm not here to give you grades, or give out punishments

INTRODUCTION TO DATA SCIENCE AND PROGRAMMING

LIAM POWER

POSTER DESIGN

POSTER DESIGN

- Today, you are just spending time designing your poster
- Find your team and get started
- If you need any help, just ask!

INTRODUCTION TO DATA SCIENCE AND PROGRAMMING

LIAM POWER

FUNCTIONS



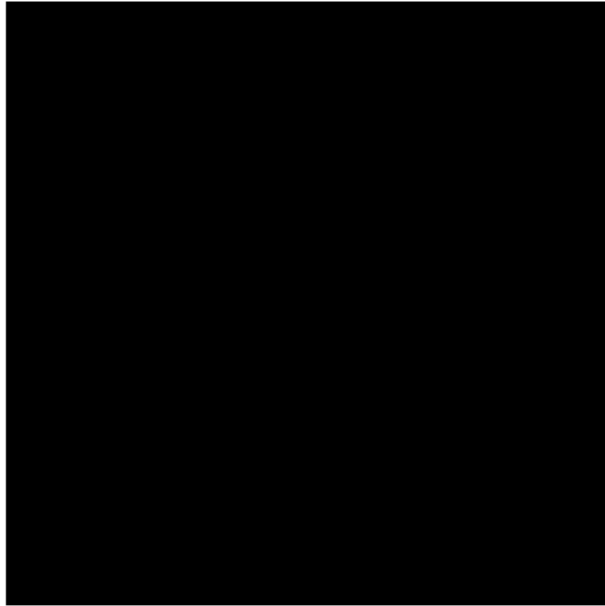
WHAT IS A
FUNCTION?

WHAT IS A FUNCTION?

- A function refers to code that performs a specific task
- You can use it to avoid repeating code and make your programs more organized
- Functions take **inputs** (arguments)
- They perform some actions
- They can give you **outputs** (return values)

print()

Takes "Hello, world!" in



prints out "Hello, world!"



PRINT()

```
def <function_name>(<inputs>):  
    #  
    # Insert code  
    #  
    return result
```

FUNCTION STRUCTURE

FUNCTION STRUCTURE

- **def** This keyword tells the computer you are defining a function
- **<function_name>** The **name** of the function, it can be anything you like
- **<inputs>** The **input**, or parameter, the function takes
- **return** This keyword specifies the **output** of the function


```
def isEven(number):  
    if number % 2 == 0:  
        return True  
    else:  
        return False
```

```
result = isEven(5)  
print(result)
```

```
result = isEven(10)  
print(result)
```

ISEVEN()

BENEFITS OF FUNCTIONS

- Why use functions?
 - Make code reusable
 - Make programs easier to read and understand
 - Break large problems into smaller, manageable pieces

```
def check_even_odd(number):
    if number % 2 == 0:
        return f"{number} is even"
    else:
        return f"{number} is odd"

def calculate_square(number):
    return f"The square of {number} is {number ** 2}"

def check_positive_negative(number):
    if number > 0:
        return f"{number} is positive"
    else:
        return f"{number} is negative"

def process_number(number):
    print(check_even_odd(number))
    print(calculate_square(number))
    print(check_positive_negative(number))

process_number(5)
process_number(10)
process_number(-7)
```

```
number1 = 5
if number1 % 2 == 0:
    print(f"{number1} is even")
else:
    print(f"{number1} is odd")

square1 = number1 ** 2
print(f"The square of {number1} is {square1}")

if number1 > 0:
    print(f"{number1} is positive")
else:
    print(f"{number1} is negative")

number2 = 10
if number2 % 2 == 0:
    print(f"{number2} is even")
else:
    print(f"{number2} is odd")

square2 = number2 ** 2
print(f"The square of {number2} is {square2}")

if number2 > 0:
    print(f"{number2} is positive")
else:
    print(f"{number2} is negative")

number3 = -7
if number3 % 2 == 0:
    print(f"{number3} is even")
else:
    print(f"{number3} is odd")

square3 = number3 ** 2
print(f"The square of {number3} is {square3}")

if number3 > 0:
    print(f"{number3} is positive")
else:
    print(f"{number3} is negative")
```

BENEFITS OF FUNCTIONS

PRACTICE

PART I

INTRODUCTION TO DATA SCIENCE AND PROGRAMMING

LIAM POWER

DATA

WHAT IS DATA?

- **Data** is **information** that is collected, observed or measured
- It is often in the form of **numbers** or **text**
 - Numbers: the temperature of a city – 5°C, 7°C, 6°C
 - Text: customer feedback – “hated it”, “disgusted”, “angry”
- It is used to draw conclusions and **better understanding**

TYPES OF DATA

- There are 2 main types of data
- **Quantitative data:** information that can be **expressed as a number**
 - For example, height, temperature or time
- **Qualitative data:** information that **describes**
 - For example, colours, names or opinions

QUALITATIVE DATA

- There are 2 main types of qualitative data
- **Nominal Data:** consists of categories that have **no ranking**
 - For example, colours of cars, types of fruits, nationality
- **Ordinal Data:** consists of categories that have **ranking**
 - For example, customer satisfaction (satisfied, neutral, dissatisfied), spice level (mild, medium, hot), movie ratings (1 star, 2 stars, ... 5 stars)

QUANTITATIVE DATA

- There are 2 main types of quantitative data
- **Discrete data:** consists of **distinct values** that can't be broken down
- For example, number of students, number of cars in a car park
- **Continuous data:** consists of **values within a range**
- For example, height of a person, time to complete a task, temperature in a city

DATA SOURCES

- **Primary sources** are **original**; they have gathered the data themselves
- For example, experiments, interviews, surveys
- **Secondary sources** are sources **based on other people's work**
- For example, textbooks, teachers, biographies

DATA SOURCES

- Positives of Primary Sources
 - **Accurate**, correct
 - **Relevant** and specific
 - **Current** and up-to-date
 - Full **Control**
- Positives of Secondary Sources
 - **Saves time** and work
 - **Easy access**
 - Good for **starting research**

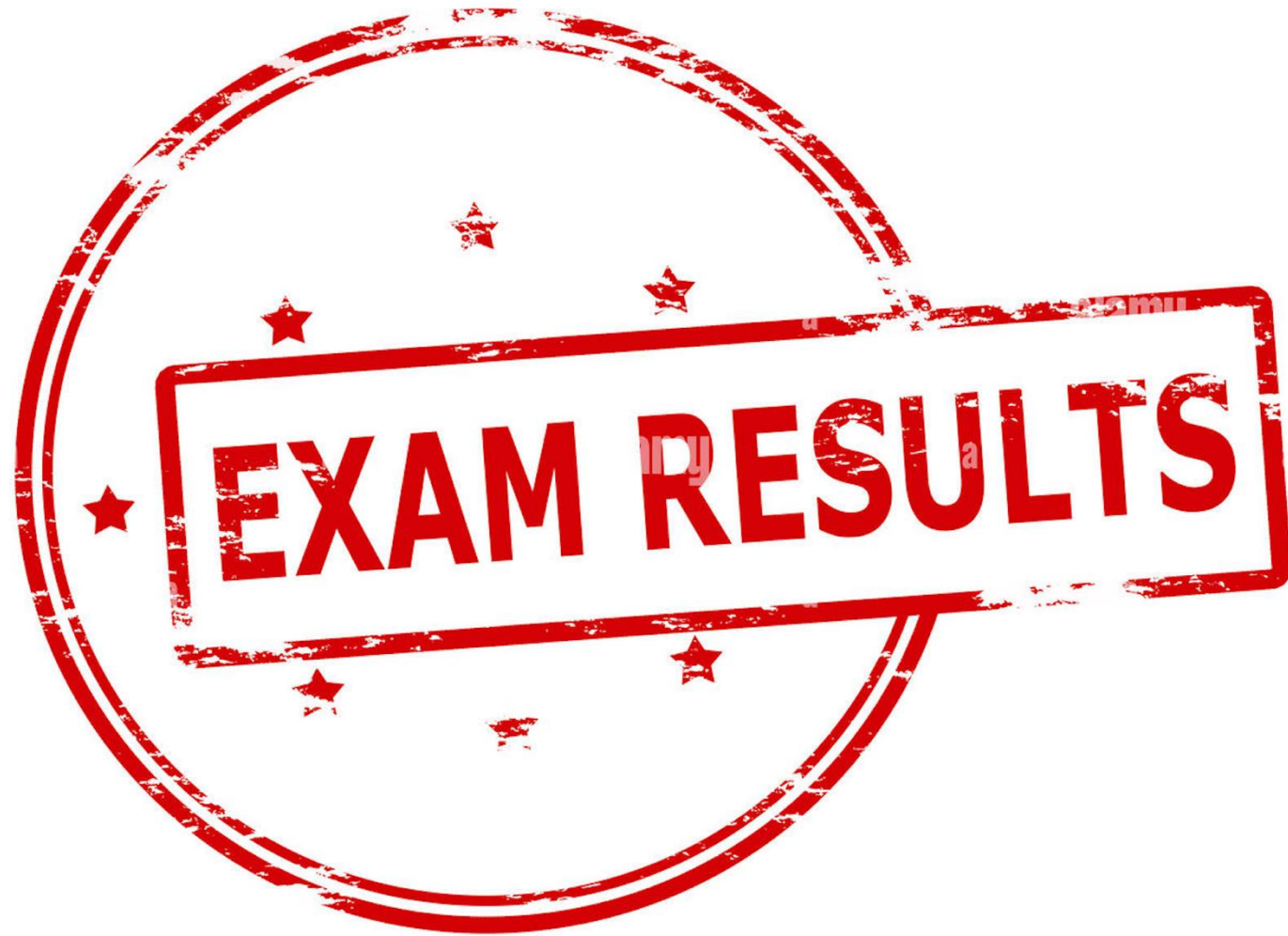
PRACTICE

PART I

EXPERIMENTS

EXPERIMENTS

- **Experiments** involve **planning how to collect data** in a way that provides a **clear answer** to a question
1. Choose a Question
 2. Build a Hypothesis
 3. Identify Variables
 4. Create Experimental Groups
 5. Conducting the Experiment
 6. Collecting the Data



EXPERIMENTS

I. CHOOSE A QUESTION

- The first step is selecting a specific **question** that you want to answer
- What makes a good research question?
 - It should be **clear** and focus on an issue
 - It should be **measurable**, so that you can collect data to answer
- For example, “does increasing the amount study time improve exam scores among high school students?”

2. BUILD A HYPOTHESIS

- A **hypothesis** is a **prediction** about what you expect to happen during the experiment
- It is an **educated guess** based on existing knowledge
- For example, “If secondary school students spend 1 extra hour studying each week, their exam scores will increase by an average of 5%”

3. IDENTIFY VARIABLES

- **Variables** are the elements that change in the experiment and that we want to measure
- There are 3 types of variables:
 - **Independent** variable: the variable **you want to change** to observe its effect
 - For example, the number of hours spent studying

3. IDENTIFY VARIABLES

- **Dependent** variable: the effect that is **measured**
- For example, exam scores
- **Control** variable: factors that are kept **constant**
- For example, the same subject for all students

4. CREATING EXPERIMENT GROUPS

- To measure the impact of the independent variable, you need to divide subjects **into groups**
- One group will receive the experimental treatment (the **experimental group**) and the other will not (the **control group**)
- It is important to assign subjects randomly to each group to **avoid bias** and to make the group as similar as possible
- For example, mixing students with good grades and poor grades

5. CONDUCTING THE EXPERIMENT

- Following the experimental design, you can conduct the actual experiment, which involves:
 - Setting a timeline
 - Monitoring variables
 - **Document everything**
- For example, ensure the extra study hours are tracked, and that both groups take the same exam under the same conditions

DATA COLLECTION

- **Data collection** is the process of **measuring the dependent variable** and **recording the results**
 - It is important to use the **same method** to collect data from both groups to ensure consistency
 - **Sample size**: collect data from a large enough group to ensure that results are valid and **not due to random chance**
- For example, collect exam scores from both groups

PRACTICE

PART 2

INTRODUCTION TO DATA SCIENCE AND PROGRAMMING

LIAM POWER

SELECTION

WHAT IS SELECTION?

- In programming, **selection** refers to a computer's process of **making decisions** based on rules, also known as **conditions**
- Selection allows programs to make decisions and **respond to different inputs**
- For example, when it's below 10°C, a computer might tell you “It is cold”, or if it's above 30°C, it might tell you “It is hot”

<	less than
<=	less than or equal to
>	greater than
>=	greater than or equal to
!=	not equal to
==	is equal to

BOOLEANS

- **Booleans are always either true or false**
- Booleans can be computed using **relational operators**

```
myAge = int(input("What is your age "))  
canDrive = (myAge >= 18)  
isTeenager = (myAge >= 13)
```

BOOLEANS

IF STATEMENTS

- **If** statements allow the computer to **select** what to do
- We give the **if statement** a boolean, which is the rule the computer must follow
- If the rule is not followed, then the computer does the **else statement**

```
myAge = int(input("What is your age "))  
canDrive = (myAge >= 18)  
  
if canDrive:  
    print("You can drive")  
else:  
    print("You cannot drive")
```

IF STATEMENTS

“AND” AND “OR” OPERATOR

- The **and** operator is used when you want **all conditions to be true** for the entire expression to evaluate as True
 - If any one of the conditions is False, the whole expression becomes False
- The **or** operator is used when you want **at least one condition to be true** for the entire expression to evaluate as True
 - The whole expression becomes False only if all conditions are False


```
age = 25
hasTicket = True

if age >= 18 and hasTicket:
    print("You can enter the event")
else:
    print("You cannot enter")

age = 16
hasPermission = True

if age >= 18 or hasPermission:
    print("You can enter the event")
else:
    print("You cannot enter")
```

IF STATEMENTS

ELIF STATEMENTS

- In Python, **elif** is short for “else if” – it is used in conditional statements when you want to test multiple rules **sequentially**
- After an initial if condition, elif allows you to check additional conditions if the previous ones are not met

```
x = 10
```

```
if x > 10:
```

```
    print("x is greater than 10")
```

```
elif x == 10:
```

```
    print("x is equal to 10")
```

```
else:
```

```
    print("x is less than 10")
```

IF STATEMENTS

PRACTICE

Part I

ITERATION

WHAT IS ITERATION?

- **Iteration** means **repeating** an action multiple times
- In programming, iteration allows us to repeat a set of instructions or tasks until a **condition is met** (while loops) or **for a specific number of times** (for loop)
- Think of a printer making multiple copies of a page

WHILE LOOPS

- A **while loop** is used when you want to keep going until a **certain condition becomes false**
 - It involves the use of a **boolean**
- It is used when **you don't know** how many times something should repeat

```
password = input("Enter your password (at least 6 characters): ")

while len(password) < 6:
    print("Password too short!")
    password = input("Please enter a password with at least 6 characters: ")

print("Password accepted!")
```

WHILE LOOPS

FOR LOOPS

- A **for loop** is used when you know exactly how many times you want to repeat an action
- It allows you to go through a sequence, stop at each **individual element** and perform actions for each element in the sequence
- If I wanted to print something out 5 times, I would use a for loop

FOR LOOPS

- An **index**, or `i`, is used to access individual elements in the collection
- With a for loop, the last number in the range is **excluded**
- **range()** is written as `range(start, stop)`
- It can also be written as `(start, stop, step)`

```
myName = "Liam"
```

```
for index in range(1, 6):  
    print(myName)
```

```
myAge = 20
```

```
for index in range(1, 6, 2):  
    print(myAge)
```

FOR LOOPS

BREAK AND CONTINUE

- You can stop a loop early using the **break** statement
- You can skip the current iteration using the **continue** statement
- These can be used to end following a condition has been met (break) or skip a specific iteration (continue)

```
for i in range(1, 6):  
    if i == 3:  
        break  
    print(i)
```

```
for i in range(1, 6):  
    if i == 3:  
        continue  
    print(i)
```

BREAK AND
CONTINUE

PRACTICE

Part 2

INTRODUCTION TO DATA SCIENCE AND PROGRAMMING

Liam Power

DATA SCIENCE



WHAT IS DATA
SCIENCE?

WHAT IS DATA SCIENCE?

- What subjects do students enjoy the most?
- How do students prefer to learn?
- Do students feel encouraged to ask questions during the lesson?
- How well are students performing on assessments?
- What feedback do students have about lessons?

DATA SCIENCE

- Data science is a **discipline** of science
- It focuses on the process of using data to **understand a topic** and **make decisions**
- It involves
 - Gathering data – **data collection**
 - Displaying data – **data visualisation**

```
001111110000001111100100110001101110010100111111111101
1110111111000011101011111101100011110111111010101100110
1101111100001110001101010010111010011111000110001000110
1100011101111011111110111111111010001111110111011111110
11111001101110111110001111011101011101000111111101111110
110011011111101100010010000001011101110111011111111111
11111000111111000111111111111110011110011111100101111111
0111111011001110111110111111101011111101111111101100111010
1111111110001111111001010010100011111011110110100111101
0001111111111011000101000011110010000000111100100100011
10111110011111111101010111111000101110111000001001111110
0111000011110111011111001111110011111111001100000110111
1011101101000010011001100011101110000110010000001100111
1101100110001010111101101000111110100011010111110010111
1111111110011000111100111101010000110100111000100100010
11101000010111001111111000001111101111111011110101111100
11100100110101111111111001111111111111111111001010111111
0000000001111111100001100111101100100011011011010110001
010001001111111100111111111111110011111000111110011110101
010001111001001111111100001011110111001101111110110011111
```

PROGRAMMING

PROGRAMMING

- A **programming language** is a special **set of rules** and instructions that allow humans to **communicate with computers**
- We can give them **commands** to perform specific tasks
- Using programming languages, we can complete tasks at lightning-fast speeds
- A **program** refers to code that does a task

PYTHON

- The programming language that we will be using is Python
- Why is Python a good language to learn?
 1. It is the most popular language in the world
 2. It plays an important part in **AI**
 3. It is **easy to understand**

```
print("Hello, world!")
```

```
.global _start
.align 2

_start:
    b _printf
    b _terminate

_printf:
    mov X0, #1
    adr X1, helloworld
    mov X2, #12
    mov X16, #4
    svc 0

_reboot:
    mov X0, #1
    mov X16, #55
    svc 0

_terminate:
    mov X0, #0
    mov X16, #1
    svc 0

helloworld: .ascii "hello world\n"
```

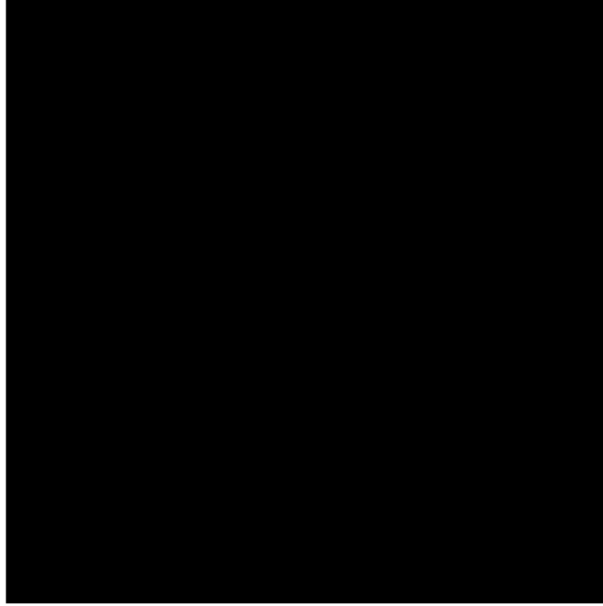
PYTHON

INTRODUCTION TO FUNCTIONS

- To help us solve problems, we will be making use of two functions
 - `print()` – this lets us give information **out from a computer**
 - `input()` – this lets us take information **in from a user**
- `print()` and `input()` are called **functions**
- Functions refers to **code** that performs a specific task

`print()`

Takes "Hello, world!" in



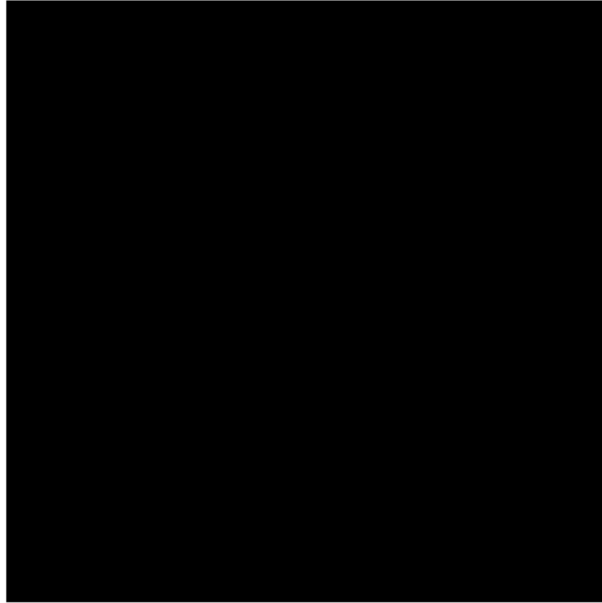
prints out "Hello, world!"



INTRODUCTION
TO FUNCTIONS

input()

Asks "What is your age?"



stores the input as a
string



INPUT()

INTRODUCTION TO FUNCTIONS

- A program can contain **functions** – each function doing a specific task that helps to complete the overall goal of the program
- For example, a program that records student information could have:
 - A function to record names
 - A function to record grades

ASSIGNMENT

WHAT IS ASSIGNMENT?

- **Assignment** is when give a value to a variable – you are **assigning** it a value
- A **variable** in programming is a **container** used to store **values** that can changed or be used later in a program
- They are called variables because you can **change**, or vary, what they store

VARIABLES

- You can name a variable **anything**
- When you assign a value to a variable, the variable must be on the **left**
- We write the names of variables using **lowerCamelCase**, to help us read the variable's name

```
myAge = 20
myName = "Liam"
gameOver = False
price = 19.99

exampleofvariablenamesinpython = "name"
examplesOfVariableNamesInPython = "name"
```

VARIABLES

TYPES

- **Types** are **labels** that tell the computer **what kind of data** we are giving it
- Every language uses types, but different programming languages might have different types

TYPES

- An **int** is short for integer – an integer is **whole positive or negative number** e.g. -113, -8, 35, 100000000
- A **String** is a sequence of characters e.g. “Hello, world!”, “my name is Liam”
 - It must be surrounded by **quotation marks**, “”
- A **boolean** can either be **true** or **false**
- A **float** is a number with a **decimal point** e.g. 123.45, -4.12, 3.1415

```
# A comment is any code that is ignored by the computer  
# Anything written after this hashtag is a comment
```

```
myAge = 20  
    # Type: int
```

```
myName = "Liam"  
    # Type: String
```

```
gameOver = False  
    # Type: boolean
```

```
price = 19.99  
    # Type: float
```

TYPES

TYPES

- You can use the **type()** function to check the type of a variable
- You can convert data from one type to another by **casting** it – wrap that variable in the type you want

```
# type() function

myName = "Liam"
myGrade = 73
pi = 3.1415
amIDone = False

print(type(myName))
print(type(myGrade))
print(type(pi))
print(type(amIDone))

# Casting

myAge = int(input("What is your age? ")) # Example of casting
birthYear = 2024 - myAge
print("You were born in " + str(birthYear)) # Example of casting
```

TYPES

Arithmetic Operators

Operator	Meaning	Example
+	Addition	$4 + 7 \longrightarrow 11$
-	Subtraction	$12 - 5 \longrightarrow 7$
*	Multiplication	$6 * 6 \longrightarrow 36$
/	Division	$30 / 5 \longrightarrow 6$
%	Modulus	$10 \% 4 \longrightarrow 2$
//	Quotient	$18 // 5 \longrightarrow 3$
**	Exponent	$3 ** 5 \longrightarrow 243$

ARITHMETIC
OPERATORS

ARITHMETIC OPERATORS

- The **modulus** operator (%) returns the remainder of a division between two numbers
- The **quotient** operator (//) gives the number of times one number can be divided by another without considering the remainder
- The **exponent** operator (**) raises a number, called the **base**, to the power of another number, called the **exponent**

PRACTICE